

桂林理工大学

实验报告

班级 软件工程 3 班

学号 3242052051355

姓名 洪耀东

实验名称 实验一 安装环境、简单编程与初识数据库

日期 2026 年 6 月 14 日

一、实验目的：

1. 掌握数据库运行环境和客户端工具的基本使用方法。
2. 掌握简单 SQL 程序设计，包括递归查询、数学计算和基础函数使用。
3. 创建订单管理数据库 OrderDB，理解主键、外键和基础数据初始化过程。

二、实验环境：

1. Windows，本机 MySQL80 服务。
2. MySQL 8.0.41，mysql 命令行客户端。
3. PowerShell、Python、PyMySQL、python-docx。

三、实验内容：

1.1 数据库环境检查

在已配置好的 MySQL 环境中连接 root 用户，查看 MySQL 版本和当前数据库，确认数据库服务可用。

1.2 SQL 简单编程

完成最大公约数、最小公倍数、水仙花数和分数数列前 20 项求和。

1.3 OrderDB 数据库初始化

创建商品分类、商品、员工、客户、订单主表和订单明细表，并插入实验数据。

四、实验练习

1.1 源码文件

D:\sandbox\MYSQL_homework\sql\01_orderdb_setup_and_programming.sql

1.2 运行结果

脚本成功创建 OrderDB，输出 MySQL 版本、表清单、程序题结果和基础数据量。

运行截图 1.1:

实验1 安装环境、简单编程与OrderDB初始化

源码与真实运行输出截图

代码: sql/01_orderdb_setup_and_programming.sql SET NAMES utf8mb4; DROP DATABASE IF EXISTS OrderDB; CREATE DATABASE OrderDB DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci; USE OrderDB; CREATE TABLE ProductClass (classNo CHAR(3) PRIMARY KEY, className VARCHAR(40) NOT NULL UNIQUE) ENGINE=InnoDB; CREATE TABLE Product (productNo CHAR(9) PRIMARY KEY, productName VARCHAR(80) NOT NULL, classNo CHAR(3) NOT NULL, productPrice DECIMAL(10,2) NOT NULL CHECK (productPrice > 0), quantity INT NOT NULL DEFAULT 0 CHECK (quantity >= 0), lowQuantity INT NOT NULL DEFAULT 5 CHECK (lowQuantity >= 0), CONSTRAINT fk_product_class FOREIGN KEY (classNo) REFERENCES ProductClass(classNo)) ENGINE=InnoDB; CREATE TABLE Employee (employeeNo CHAR(8) PRIMARY KEY, employeeName VARCHAR(30) NOT NULL, sex CHAR(1) NOT NULL CHECK (sex IN ('M','F')), department VARCHAR(30) NOT NULL, headship VARCHAR(30) NOT NULL, hireDate DATE NOT NULL, birthday DATE NULL, salary DECIMAL(10,2) NOT NULL CHECK (salary >= 0), address VARCHAR(100), CONSTRAINT chk_employee_no CHECK (employeeNo REGEXP '^E[0-9]{7}\$')) ENGINE=InnoDB; CREATE TABLE Customer (customerNo CHAR(9) PRIMARY KEY, customerName VARCHAR(80) NOT NULL, address VARCHAR(120), zip VARCHAR(10), phone VARCHAR(30)) ENGINE=InnoDB; CREATE TABLE OrderMaster (orderNo CHAR(9) PRIMARY KEY, customerNo CHAR(9) NOT NULL, employeeNo CHAR(8) NULL, orderDate DATE NOT NULL, orderSum DECIMAL(12,2) NOT NULL DEFAULT 0, CONSTRAINT fk_ordermaster_customer FOREIGN KEY (customerNo) REFERENCES Customer(customerNo) ON UPDATE CASCADE ON DELETE RESTRICT, CONSTRAINT fk_ordermaster_employee FOREIGN KEY (employeeNo) REFERENCES Employee(employeeNo) ON UPDATE CASCADE ON DELETE SET NULL) ENGINE=InnoDB; CREATE TABLE OrderDetail (orderNo CHAR(9) NOT NULL, productNo CHAR(9) NOT NULL, quantity INT NOT NULL CHECK (quantity > 0), price DECIMAL(10,2) NOT NULL CHECK (price > 0), PRIMARY KEY (orderNo, productNo), CONSTRAINT fk_detail_order FOREIGN KEY (orderNo) REFERENCES OrderMaster(orderNo) ... -- 省略中间 110 行 SELECT '实验1：求252和105的最大公约数与最小公倍数' AS section; WITH RECURSIVE gcd_calc(a,b) AS (SELECT 252, 105 UNION ALL SELECT b, MOD(a,b) FROM gcd_calc WHERE b <> 0) SELECT 252 AS first_number, 105 AS second_number, a AS gcd_value, 252 * 105 / a AS lcm_value FROM gcd_calc WHERE b = 0; SELECT '实验1：0到999之间的水仙花数' AS section; WITH RECURSIVE nums(n) AS (SELECT 0 UNION ALL ...	运行输出: outputs/01_orderdb_setup_and_programming.txt +-----+ section +-----+ 实验1：MySQL运行环境与OrderDB初始化 +-----+ +-----+ mysql_version current_database +-----+ 8.0.41 orderdb +-----+ +-----+ Tables_in_orderdb +-----+ customer employee orderdetail ordermaster product productclass +-----+ +-----+ section +-----+ 实验1：求252和105的最大公约数与最小公倍数 +-----+ +-----+ first_number second_number gcd_value lcm_value +-----+ 252 105 21 1260.0000 +-----+ +-----+ section +-----+ 实验1：0到999之间的水仙花数 +-----+ +-----+ narcissistic_numbers +-----+ 0, 1, 153, 370, 371, 407 +-----+ +-----+ section +-----+ 实验1：分数数列前20项之和 +-----+ +-----+ sum_first_20_terms +-----+ 32.660261 +-----+ +-----+ section +-----+ 实验1：OrderDB基础数据量 +-----+ +-----+ product_classes products employees customers orders order_details +-----+ 5 12 8 9 15 32 +-----+ ...
---	--

五、实验心得

本次实验把原指导书中的 SQL Server 环境内容迁移到 MySQL 实现，熟悉了 MySQL 数据类型、CHECK 约束、递归 CTE 和外键约束的写法。

桂林理工大学

实验报告

班级 软件工程 3 班

学号 3242052051355

姓名 洪耀东

实验名称 实验二 SQL 查询

日期 2026 年 6 月 14 日

一、实验目的：

1. 掌握 SELECT 语句、条件查询、聚合函数和分组统计。
2. 掌握多表连接、子查询、EXISTS 和复杂汇总查询。

二、实验环境：

1. Windows，本机 MySQL80 服务。
2. MySQL 8.0.41，OrderDB 数据库。
3. mysql 命令行客户端、PowerShell。

三、实验内容：

1.1 单表查询与聚合

完成客户、员工、订单和订单明细相关的条件查询、排序、聚合和分组统计。

1.2 多表查询

完成内连接、左外连接、右外连接、UNION 模拟完整外连接和子查询。

1.3 复杂查询

完成未订货客户、订购多种商品客户、销售员业绩、EXISTS 相关查询等任务。

四、实验练习

1.1 源码文件

D:\sandbox\MYSQL_homework\sql\02_orderdb_queries.sql

1.2 运行结果

实验 2-1、2-2、2-3 查询均成功执行，结果日志保存在 outputs 目录中。

运行截图 2.1：

实验2 SQL查询

源码与真实运行输出截图

代码: sql/02_orderdb_queries.sql

```
SET NAMES utf8mb4;
USE OrderDB;
UPDATE OrderMaster om
JOIN (
  SELECT orderNo, SUM(quantity * price) AS sum2
  FROM OrderDetail
  GROUP BY orderNo
) d ON om.orderNo = d.orderNo
SET om.orderSum = d.sum2;
SELECT '实验2-1: 单表查询与聚合' AS section;
SELECT '2-1(1) 名称中含有“有限”的客户' AS question;
SELECT customerName, address
FROM Customer
WHERE customerName LIKE '%有限%';
SELECT '2-1(2) 姓张且姓名最后一字为梅的员工' AS question;
SELECT employeeNo, employeeName, department, headship
FROM Employee
WHERE employeeName LIKE '张%梅';
SELECT '2-1(3) 住址含上海或南昌的员工' AS question;
SELECT employeeName,
       department,
       headship,
       address,
       IFNULL(DATE_FORMAT(birthday, '%Y-%m-%d'), '不详') AS birthday,
       CASE sex WHEN 'M' THEN '男' ELSE '女' END AS sex
FROM Employee
WHERE address LIKE '%上海%' OR address LIKE '%南昌%';
SELECT '2-1(4) 客户编号不在C20200003到C20210002之间的客户' AS question;
SELECT customerNo, customerName, address
FROM Customer
WHERE customerNo NOT BETWEEN 'C20200003' AND 'C20210002';
SELECT '2-1(5) 订单金额最高的前10%订单' AS question;
WITH ranked_orders AS (
  SELECT om.*,
         ROW_NUMBER() OVER (ORDER BY orderSum DESC) AS rn,
         COUNT(*) OVER () AS total_count
  FROM OrderMaster om
)
SELECT orderNo, customerNo, employeeNo, orderDate, orderSum
FROM ranked_orders
WHERE rn <= CEIL(total_count * 0.10)
ORDER BY orderSum DESC;
SELECT '2-1(6) 一共销售了几种商品' AS question;
SELECT COUNT(DISTINCT productNo) AS sold_product_count
FROM OrderDetail;
SELECT '2-1(7) 每种商品销售数量、平均单价和总金额' AS question;
SELECT productNo,
       SUM(quantity) AS total_quantity,
       ROUND(AVG(price), 2) AS avg_price,
       SUM(quantity * price) AS total_amount
FROM OrderDetail
GROUP BY productNo
ORDER BY total_amount DESC;
SELECT '2-1(8) 每个客户2020年3月订单总金额' AS question;
SELECT customerNo, SUM(orderSum) AS march_2020_amount
--
-- 省略中间 205 行 --
--
LIMIT 1;
SELECT '2-3(8) 用EXISTS查找没有订货记录的客户' AS question;
SELECT c.customerName
FROM Customer c
WHERE NOT EXISTS (
  SELECT 1 FROM OrderMaster om WHERE om.customerNo = c.customerNo
);
SELECT '2-3(9) 至少包含“手环”商品的订单明细' AS question;
SELECT om.orderNo, c.customerName, p.productName, od.quantity, od.price
FROM OrderMaster om
JOIN Customer c ON om.customerNo = c.customerNo
JOIN OrderDetail od ON om.orderNo = od.orderNo
JOIN Product p ON od.productNo = p.productNo
WHERE EXISTS (
  SELECT 1
  FROM OrderDetail od2
  JOIN Product p2 ON od2.productNo = p2.productNo
  WHERE od2.orderNo = om.orderNo
  AND p2.productName LIKE '%手环%'
)
--
```

运行输出: outputs/02_orderdb_queries.txt

```
+-----+
| section |
+-----+
| 实验2-1: 单表查询与聚合 |
+-----+
+-----+
| question |
+-----+
| 2-1(1) 名称中含有“有限”的客户 |
+-----+
+-----+
| customerName | address |
+-----+
| 上海海天有限公司 | 上海市浦东新区 |
| 南昌红谷滩有限公司 | 南昌市红谷滩新区 |
| 桂林电子有限公司 | 桂林市七星区 |
| 北京智联科技有限公司 | 北京市海淀区 |
| 合生源有限公司 | 广州市天河区 |
| 广州南方有限集团 | 广州市越秀区 |
+-----+
+-----+
| question |
+-----+
| 2-1(2) 姓张且姓名最后一字为梅的员工 |
+-----+
+-----+
| employeeNo | employeeName | department | headship |
+-----+
| E2019002 | 张小梅 | 业务科 | 业务员 |
| E2020004 | 张梅 | 业务科 | 业务员 |
+-----+
+-----+
| question |
+-----+
| 2-1(3) 住址含上海或南昌的员工 |
+-----+
... 省略中间 442 行 ...
+-----+
| C20200004 | P20200001 | 1 | 1450.00 |
| C20200004 | P20200003 | 4 | 899.00 |
| C20200004 | P20210002 | 6 | 239.00 |
| C20200004 | P20210004 | 2 | 1299.00 |
| C20210001 | P20210001 | 6 | 299.00 |
| C20210001 | P20210004 | 1 | 1299.00 |
| C20210001 | P20210005 | 10 | 199.00 |
| C20210001 | P20210006 | 3 | 899.00 |
| C20210002 | P20200005 | 10 | 399.00 |
| C20210002 | P20210001 | 15 | 299.00 |
| C20210002 | P20210002 | 15 | 239.00 |
| C20210002 | P20210005 | 15 | 199.00 |
| C20210003 | P20200002 | 3 | 2499.00 |
| C20210003 | P20210001 | 5 | 299.00 |
| C20210004 | P20200001 | 2 | 1450.00 |
| C20210004 | P20210006 | 2 | 899.00 |
+-----+
+-----+
| question |
+-----+
| 2-3(7) 业绩最好的业务员 |
+-----+
+-----+
| employeeNo | employeeName | total_sales |
+-----+
| E2019002 | 张小梅 | 23374.00 |
+-----+
+-----+
| question |
+-----+
| 2-3(8) 用EXISTS查找没有订货记录的客户 |
+-----+
+-----+
| customerName |
+-----+
| 杭州未来公司 |
+-----+
+-----+
| question |
+-----+
...
```

五、实验心得

复杂查询的关键是先明确查询粒度，再选择连接、分组或子查询。WHERE 用于分组前过滤，HAVING 用于聚合后过滤。

桂林理工大学

实验报告

班级 软件工程 3 班

学号 3242052051355

姓名 洪耀东

实验名称 实验三 数据库与数据表定义、索引视图和数据更新
日

日期 2026 年 6 月 14

一、实验目的：

1. 掌握数据库、基本表、主键、外键和约束的定义方法。
2. 掌握索引、视图的创建、查询和删除。
3. 掌握 INSERT、UPDATE、DELETE 及其与约束之间的关系。

二、实验环境：

1. Windows，本机 MySQL80 服务。
2. MySQL 8.0.41，BookDB 数据库。
3. mysql 命令行客户端、PowerShell。

三、实验内容：

1.1 BookDB 数据库定义

创建图书分类、出版社、图书、读者、借阅五张表，并设置主键和外键。

1.2 索引与视图

创建出版社号、身份证号、工作单位等索引，创建 BookView、BorrowView、ReaderView 等视图。

1.3 数据更新

完成读者工作单位修改、图书价格调整、库存调整和借阅记录删除等操作。

四、实验练习

1.1 源码文件

D:\sandbox\MYSQL_homework\sql\03_bookdb_definition_indexes_views_updates.sql

1.2 运行结果

BookDB 创建成功，索引、视图、查询和更新语句均执行成功。

运行截图 3.1:

实验3 BookDB定义、索引、视图与更新

源码与真实运行输出截图

代码: sql/03_bookdb_definition_indexes_views_updates.sql

```
SET NAMES utf8mb4;
DROP DATABASE IF EXISTS BookDB;
CREATE DATABASE BookDB DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
USE BookDB;
CREATE TABLE BookClass (
  classNo CHAR(3) PRIMARY KEY,
  className VARCHAR(40) NOT NULL UNIQUE
) ENGINE=InnoDB;
CREATE TABLE Publisher (
  publisherNo CHAR(4) PRIMARY KEY,
  publisherName VARCHAR(80) NOT NULL,
  address VARCHAR(120),
  phone VARCHAR(30)
) ENGINE=InnoDB;
CREATE TABLE Book (
  bookNo CHAR(10) PRIMARY KEY,
  classNo CHAR(3) NOT NULL,
  publisherNo CHAR(4) NOT NULL,
  bookName VARCHAR(100) NOT NULL,
  author VARCHAR(60),
  price DECIMAL(8,2) NOT NULL CHECK (price BETWEEN 10 AND 80),
  stock INT NOT NULL DEFAULT 0 CHECK (stock >= 0),
  CONSTRAINT fk_book_class FOREIGN KEY (classNo) REFERENCES BookClass(classNo),
  CONSTRAINT fk_book_publisher FOREIGN KEY (publisherNo) REFERENCES Publisher(publisherNo)
) ENGINE=InnoDB;
CREATE TABLE Reader (
  readerNo CHAR(8) PRIMARY KEY,
  readerName VARCHAR(40) NOT NULL,
  sex CHAR(1) NOT NULL CHECK (sex IN ('男','女')),
  identifyCard CHAR(18) NOT NULL UNIQUE,
  workUnit VARCHAR(100),
  maxBorrowCount INT NOT NULL DEFAULT 0 CHECK (maxBorrowCount BETWEEN 0 AND 8),
  currentBorrowCount INT NOT NULL DEFAULT 0 CHECK (currentBorrowCount BETWEEN 0 AND 8),
  borrowPeriodDays INT NOT NULL DEFAULT 30
) ENGINE=InnoDB;
CREATE TABLE Borrow (
  readerNo CHAR(8) NOT NULL,
  bookNo CHAR(10) NOT NULL,
  borrowDate DATE NOT NULL DEFAULT (CURRENT_DATE),
  returnDueDate DATE NOT NULL,
  actualReturnDate DATE NULL DEFAULT NULL,
  PRIMARY KEY (readerNo, bookNo, borrowDate),
  CONSTRAINT fk_borrow_reader FOREIGN KEY (readerNo) REFERENCES Reader(readerNo)
  ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_borrow_book FOREIGN KEY (bookNo) REFERENCES Book(bookNo)
  ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT chk_borrow_date CHECK (
    returnDueDate > borrowDate
    AND (actualReturnDate IS NULL OR actualReturnDate > borrowDate)
  )
) ENGINE=InnoDB;
INSERT INTO BookClass VALUES
('001','计算机类'),
('002','经济类'),
('003','外语类'),
('004','文学类'),
('005','机械类');
...
... 省略中间 134 行 ...
...
SET b.price = ROUND(b.price * 1.10, 2)
WHERE c.className = '经济类';
UPDATE Book b
JOIN (
  SELECT bookNo, COUNT(*) AS borrow_count
  FROM Borrow
  GROUP BY bookNo
  HAVING COUNT(*) > 2
) x ON b.bookNo = x.bookNo
SET b.stock = FLOOR(b.stock * 1.5);
UPDATE Reader
SET borrowPeriodDays = 90
WHERE workUnit = '兴隆股份有限公司';
...
```

运行输出: outputs/03_bookdb_definition_indexes_views_updates.txt

```
+-----+
| section |
+-----+
| 实验3-1: BookDB数据库与5张基本表创建完成 |
+-----+
+-----+
| Tables in bookdb |
+-----+
| book |
| bookclass |
| borrow |
| publisher |
| reader |
+-----+
+-----+
| section |
+-----+
| 实验3-2: BookView中书名含“数据”的图书 |
+-----+
+-----+
| bookNo | bookName | author | price |
+-----+
| B201801001 | 数据库系统原理 | 王珊 | 58.00 |
| B201801002 | 数据结构 | 严蔚敏 | 45.00 |
| B201801003 | MySQL数据库应用 | 李明 | 39.00 |
| B202101005 | 数据库与数据挖掘 | 陈文伟 | 52.00 |
| B202201001 | Python数据分析 | 周志华 | 50.00 |
+-----+
+-----+
| section |
+-----+
| 实验3-2: BorrowView中2016年3月1日以前借阅的图书 |
+-----+
+-----+
| section |
+-----+
| 实验3-2: ReaderView中姓张的读者 |
+-----+
+-----+
| readerNo | readerName | sex | workUnit | borrowTimes |
+-----+
| R2020002 | 张颖 | 女 | 兴隆股份有限公司 | 3 |
+-----+
+-----+
| section |
+-----+
| 实验3-2: 合生元有限公司读者借书视图BorrowView1 |
+-----+
+-----+
| readerNo | readerName | bookNo | bookName | borrowDate | returnDueDate |
+-----+
+
| R2020001 | 欧阳 | B201801001 | 数据库系统原理 | 2020-01-10 | 2020-02-10 |
| R2020001 | 欧阳 | B201801002 | 数据结构 | 2020-02-10 | 2020-03-10 |
| R2020001 | 欧阳 | B202001004 | 大学英语 | 2020-03-11 | 2020-04-11 |
| R2021001 | 王涛 | B201801001 | 数据库系统原理 | 2021-01-10 | 2021-02-10 |
| R2021001 | 王涛 | B201801002 | 数据结构 | 2021-01-12 | 2021-02-12 |
| R2021001 | 王涛 | B201801003 | MySQL数据库应用 | 2021-01-14 | 2021-02-14 |
+-----+
+
+-----+
| section |
+-----+
| 实验3-3: 数据更新操作 |
+-----+
+-----+
| section |
+-----+
| 实验3-3: 更新后数据统计 |
+-----+
+-----+
| reader_count | borrow_count | economic_book_count |
+-----+
| 4 | 5 | 4 |
+-----+
```

五、实验心得

视图能简化复杂连接查询，索引能提高检索效率。进行数据更新前应先分析外键和业务约束，避免破坏数据一致性。

桂林理工大学

实验报告

班级 软件工程 3 班

学号 3242052051355

姓名 洪耀东

实验名称 实验四 游标、存储过程与触发器

日期 2026 年 6 月 14 日

一、实验目的：

1. 掌握 MySQL 游标和存储过程的创建、执行和调用方法。
2. 掌握触发器在安全控制、数据校验和库存维护中的应用。

二、实验环境：

1. Windows，本机 MySQL80 服务。
2. MySQL 8.0.41，OrderDB 数据库。
3. mysql 命令行客户端、PowerShell。

三、实验内容：

1.1 游标与存储过程

使用游标统计订单金额，编写员工编号生成、大客户热销商品、客户销售汇总和发票输出存储过程。

1.2 触发器

编写客户删除控制、薪水修改审计、员工日期校验和订单明细库存维护触发器。

四、实验练习

1.1 源码文件

D:\sandbox\MYSQL_homework\sql\04_orderdb_procedures_triggers.sql

1.2 运行结果

存储过程调用成功，触发器生成薪水审计记录，并在插入订单明细后自动更新库存和订单金额。

运行截图 4.1:

源码与真实运行输出截图

... 省略中间 213 行 ...

section				
实验4-1: 游标与存储过程				
orderNo orderSum				
O20200001 5341.00				
O20200002 9900.00				
O20200003 4245.00				
O20200004 4048.00				
O20200005 4491.00				
generated_employee_no				
E2020006				
customerNo customerName		productNo productName		sale_quantity sale_amount
C20200001 上海海天有限公司		P20210007 酷睿四核i7散热器		20 3180.00
C20200001 上海海天有限公司		P20200004 金士顿16G内存		13 4277.00
C20200001 上海海天有限公司		P20210005 TP-LINK千兆路由器		4 796.00
C20200002 南昌红谷有限责任公司		P20210002 小米手环7		20 4780.00
C20200002 南昌红谷有限责任公司		P20210003 罗技无线鼠标		20 2580.00
C20200002 南昌红谷有限责任公司		P20210001 华为手环6		10 2990.00
C20200003 桂林电子有限公司		P20210003 罗技无线鼠标		22 2838.00
C20200003 桂林电子有限公司		P20200005 西部数据1T硬盘		8 3192.00
C20200003 桂林电子有限公司		P20200003 华硕主板Z270		3 2697.00
C20210001 合生元有限公司		P20210005 TP-LINK千兆路由器		10 1990.00
C20210001 合生元有限公司		P20210001 华为手环6		6 794.00
C20210001 合生元有限公司		P20210006 戴尔显示器24寸		3 2697.00
C20210002 广州南方有限集团		P20210001 华为手环6		15 485.00
C20210002 广州南方有限集团		P20210002 小米手环7		15 3585.00
C20210002 广州南方有限集团		P20210005 TP-LINK千兆路由器		15 2985.00
customerNo customerName		address		totalOrderAmount reportDate
C20200002 南昌红谷有限责任公司		南昌市红谷滩新区		15348.00 2026-06-14
productNo productName		saleQuantity saleAmount		
P20200002 酷睿四核i7-7700k		2 4998.00		
P20210001 华为手环6		10 2990.00		
P20210002 小米手环7		20 4780.00		
P20210003 罗技无线鼠标		20 2580.00		
invoiceNo customerName		orderDate employeeName		orderSum printDate

游标适合逐行处理过程化任务，触发器适合把审计、校验和库存维护封装到数据库内部，减少应用层遗漏。

桂林理工大学

实验报告

班级 软件工程 3 班

学号 3242052051355

姓名 洪耀东

实验名称 实验五 安全性定义与检查

日期 2026 年 6 月 14 日

一、实验目的：

1. 掌握数据库用户、角色和权限的创建、授权、回收和检查。
2. 理解 MySQL 权限模型与 SQL Server 用户模型之间的差异。

二、实验环境：

1. Windows，本机 MySQL80 服务。
2. MySQL 8.0.41，OrderDB 数据库。
3. mysql 命令行客户端、PowerShell。

三、实验内容：

1.1 用户与角色创建

创建 user01、user02、user03、user05、user06、user07 等用户，并创建 r3、r5、r6 等角色。

1.2 授权与回收

完成表级权限、列级权限、角色权限、建表和建视图权限的授予与回收。

1.3 权限检查

使用 SHOW GRANTS 检查用户权限，创建测试表和视图验证权限对象。

四、实验练习

1.1 源码文件

D:\sandbox\MYSQL_homework\sql\05_security_mysql.sql

1.2 运行结果

用户、角色和授权语句执行成功，SHOW GRANTS 输出显示权限分配结果。

运行截图 5.1：

实验5 安全性定义与检查

源码与真实运行输出截图

代码: sql/05_security_mysql.sql SET NAMES utf8mb4; USE OrderDB; DROP USER IF EXISTS 'user01'@'localhost'; DROP USER IF EXISTS 'user02'@'localhost'; DROP USER IF EXISTS 'user03'@'localhost'; DROP USER IF EXISTS 'user05'@'localhost'; DROP USER IF EXISTS 'user06'@'localhost'; DROP USER IF EXISTS 'user07'@'localhost'; DROP USER IF EXISTS 'login03'@'localhost'; DROP ROLE IF EXISTS 'r3'@'localhost'; DROP ROLE IF EXISTS 'r4'@'localhost'; DROP ROLE IF EXISTS 'r5'@'localhost'; DROP ROLE IF EXISTS 'r6'@'localhost'; CREATE USER 'user01'@'localhost' IDENTIFIED BY 'p888888'; CREATE USER 'user02'@'localhost' IDENTIFIED BY 'p888888'; CREATE USER 'login03'@'localhost' IDENTIFIED BY 'p888888'; CREATE USER 'user05'@'localhost' IDENTIFIED BY 'p888888'; CREATE USER 'user06'@'localhost' IDENTIFIED BY 'p888888'; CREATE USER 'user07'@'localhost' IDENTIFIED BY 'p888888'; RENAME USER 'login03'@'localhost' TO 'user03'@'localhost'; GRANT ALL PRIVILEGES ON OrderDB.Employee TO 'user01'@'localhost', 'user02'@'localhost', 'user03'@'localhost'; GRANT SELECT ON OrderDB.Product TO 'user03'@'localhost'; GRANT SELECT (employeeNo, employeeName), UPDATE (employeeNo, employeeName) ON OrderDB.Employee TO 'user03'@'localhost'; CREATE ROLE 'r3'@'localhost'; CREATE ROLE 'r4'@'localhost'; GRANT SELECT ON OrderDB.OrderDetail TO 'r3'@'localhost'; GRANT UPDATE (price) ON OrderDB.OrderDetail TO 'r3'@'localhost'; REVOKE ALL PRIVILEGES ON OrderDB.Employee FROM 'user01'@'localhost', 'user02'@'localhost', 'user03'@'localhost'; GRANT 'r3'@'localhost' TO 'user01'@'localhost', 'user02'@'localhost'; SET DEFAULT ROLE 'r3'@'localhost' TO 'user01'@'localhost', 'user02'@'localhost'; REVOKE 'r3'@'localhost' FROM 'user02'@'localhost'; DROP ROLE 'r4'@'localhost'; GRANT CREATE, CREATE VIEW ON OrderDB.* TO 'user01'@'localhost'; DROP TABLE IF EXISTS Table_user01; CREATE TABLE Table_user01 (id INT PRIMARY KEY, note VARCHAR(40)) ENGINE=InnoDB; INSERT INTO Table_user01 VALUES (1, 'user01 table demo'); CREATE OR REPLACE VIEW View_user01 AS SELECT id, note FROM Table_user01; GRANT SELECT ON OrderDB.Table_user01 TO 'user02'@'localhost', 'user03'@'localhost'; GRANT SELECT ON OrderDB.View_user01 TO 'user02'@'localhost', 'user03'@'localhost'; DROP TABLE IF EXISTS Table1; CREATE TABLE Table1 (id INT PRIMARY KEY, content VARCHAR(40)) ENGINE=InnoDB; INSERT INTO Table1 VALUES (1, 'created for user07'); GRANT SELECT, INSERT ON OrderDB.Table1 TO 'user02'@'localhost'; DROP TABLE IF EXISTS Table2; -- ... 省略中间 11 行 ... -- CREATE OR REPLACE VIEW View1 AS SELECT t2.id, t2.content AS content2, t3.content AS content3 FROM Table2 t2 JOIN Table3 t3 ON t2.id = t3.id; GRANT SELECT ON OrderDB.Table2 TO 'user05'@'localhost', 'user06'@'localhost' WITH GRANT OPTION; GRANT SELECT ON OrderDB.Table3 TO 'user05'@'localhost', 'user06'@'localhost' WITH GRANT OPTION; GRANT SELECT ON OrderDB.View1 TO 'user05'@'localhost', 'user06'@'localhost' WITH GRANT OPTION; GRANT SELECT ON OrderDB.Table2 TO 'user07'@'localhost'; CREATE ROLE 'r5'@'localhost'; CREATE ROLE 'r6'@'localhost'; GRANT CREATE, CREATE VIEW ON OrderDB.* TO 'r5'@'localhost'; GRANT SELECT, INSERT ON OrderDB.Customer TO 'r6'@'localhost'; GRANT SELECT, UPDATE, INSERT ON OrderDB.Employee TO 'r6'@'localhost'; GRANT ALL PRIVILEGES ON OrderDB.OrderMaster TO 'r6'@'localhost'; GRANT 'r5'@'localhost' TO 'user07'@'localhost'; SET DEFAULT ROLE 'r5'@'localhost' TO 'user07'@'localhost'; DROP TABLE IF EXISTS Table4; CREATE TABLE Table4 (--	运行输出: outputs/05_security_mysql.txt +-----+ section +-----+ 实验5: MySQL安全性定义与权限检查 +-----+ +-----+ Grants for user01@localhost +-----+ GRANT USAGE ON *.* TO 'user01'@'localhost' GRANT CREATE, CREATE VIEW ON 'orderdb'.* TO 'user01'@'localhost' GRANT 'r3'@'localhost' TO 'user01'@'localhost' +-----+ +-----+ Grants for user02@localhost +-----+ GRANT USAGE ON *.* TO 'user02'@'localhost' GRANT SELECT, INSERT ON 'orderdb'.'table1' TO 'user02'@'localhost' GRANT SELECT ON 'orderdb'.'table_user01' TO 'user02'@'localhost' GRANT SELECT ON 'orderdb'.'view_user01' TO 'user02'@'localhost' +-----+ +-----+ Grants for user03@localhost +-----+ GRANT USAGE ON *.* TO 'user03'@'localhost' GRANT SELECT ON 'orderdb'.'product' TO 'user03'@'localhost' GRANT SELECT ON 'orderdb'.'table_user01' TO 'user03'@'localhost' GRANT SELECT ON 'orderdb'.'view_user01' TO 'user03'@'localhost' +-----+ +-----+ Grants for user07@localhost +-----+ GRANT USAGE ON *.* TO 'user07'@'localhost' GRANT SELECT ON 'orderdb'.'table2' TO 'user07'@'localhost' GRANT 'r5'@'localhost' TO 'user07'@'localhost' +-----+ +-----+ user host +-----+ user01 localhost user02 localhost user03 localhost user05 localhost user06 localhost user07 localhost +-----+ +-----+ section +-----+ 权限对象检查 +-----+ +-----+ TABLE_NAME +-----+ table1 table2 table3 table4 table_user01 view1 view_user01 +-----+
---	--

五、实验心得

数据库安全控制不仅是能否登录，还包括对象级、列级和角色继承等细粒度权限管理。

桂林理工大学

实验报告

班级 软件工程 3 班 学号 3242052051355 姓名 洪耀东
实验名称 实验六 完整性定义与检查 日期 2026 年 6 月 14 日

一、实验目的：

1. 掌握实体完整性、参照完整性和用户自定义完整性的定义方法。
2. 通过插入、删除、更新操作观察完整性约束的检查效果。

二、实验环境：

1. Windows，本机 MySQL80 服务。
2. MySQL 8.0.41，BookDB 数据库。
3. mysql 命令行客户端、PowerShell。

三、实验内容：

1.1 完整性约束定义

重新创建 BookDB 五张表，设置主键、外键、唯一约束、CHECK 约束和默认值。

1.2 数据插入

插入 5 条图书分类、5 条出版社、20 条图书、10 条读者和 50 条借阅记录。

1.3 完整性检查

测试重复主键、重复身份证、外键限制、日期限制、最大借书数和价格范围约束。

四、实验练习

1.1 源码文件

D:\sandbox\MYSQL_homework\sql\06_bookdb_integrity.sql

1.2 运行结果

合法操作执行成功，违反主键、外键或 CHECK 约束的操作被 MySQL 完整性机制拦截。

运行截图 6.1：

实验6 完整性定义与检查

源码与真实运行输出截图

代码: sql/06_bookdb_integrity.sql

```
SET NAMES utf8mb4;
DROP DATABASE IF EXISTS BookDB;
CREATE DATABASE BookDB DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
USE BookDB;
CREATE TABLE BookClass (
  classNo CHAR(3) PRIMARY KEY,
  className VARCHAR(40) NOT NULL UNIQUE
) ENGINE=InnoDB;
CREATE TABLE Publisher (
  publisherNo CHAR(4) PRIMARY KEY,
  publisherName VARCHAR(80) NOT NULL,
  address VARCHAR(120),
  phone VARCHAR(30)
) ENGINE=InnoDB;
CREATE TABLE Reader (
  readerNo CHAR(8) PRIMARY KEY,
  readerName VARCHAR(40) NOT NULL,
  sex CHAR(1) NOT NULL CHECK (sex IN ('男','女')),
  identifyCard CHAR(18) NOT NULL UNIQUE,
  workUnit VARCHAR(100),
  maxBorrowCount INT NOT NULL DEFAULT 0 CHECK (maxBorrowCount BETWEEN 0 AND 8),
  currentBorrowCount INT NOT NULL DEFAULT 0 CHECK (currentBorrowCount BETWEEN 0 AND 8),
  CONSTRAINT chk_reader_no CHECK (
    readerNo REGEXP '^R[0-9]{7}$'
    AND CAST(SUBSTRING(readerNo, 2, 4) AS UNSIGNED) <= 2026
  )
) ENGINE=InnoDB;
CREATE TABLE Book (
  bookNo CHAR(10) PRIMARY KEY,
  classNo CHAR(3) NOT NULL,
  publisherNo CHAR(4) NOT NULL,
  bookName VARCHAR(100) NOT NULL,
  author VARCHAR(60),
  price DECIMAL(8,2) NOT NULL CHECK (price BETWEEN 10 AND 80),
  stock INT NOT NULL DEFAULT 0 CHECK (stock >= 0),
  CONSTRAINT chk_book_no CHECK (
    bookNo REGEXP '^B[0-9]{9}$'
    AND CAST(SUBSTRING(bookNo, 2, 4) AS UNSIGNED) <= 2026
  ),
  CONSTRAINT fk_book_class_int FOREIGN KEY (classNo) REFERENCES BookClass(classNo)
  ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_book_publisher_int FOREIGN KEY (publisherNo) REFERENCES Publisher(publisherNo)
  ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;
CREATE TABLE Borrow (
  readerNo CHAR(8) NOT NULL,
  bookNo CHAR(10) NOT NULL,
  borrowDate DATE NOT NULL DEFAULT (CURRENT_DATE),
  returnDueDate DATE NOT NULL,
  actualReturnDate DATE NULL DEFAULT NULL,
  PRIMARY KEY (readerNo, bookNo, borrowDate),
  CONSTRAINT fk_borrow_reader_int FOREIGN KEY (readerNo) REFERENCES Reader(readerNo)
  ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_borrow_book_int FOREIGN KEY (bookNo) REFERENCES Book(bookNo)
  ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT chk_borrow_dates_int CHECK (
    returnDueDate > borrowDate
    AND (actualReturnDate IS NULL OR actualReturnDate > borrowDate)
  )
)
-- 省略中间 119 行 ...
--
DELIMITER ;
SELECT '实验6：完整性约束定义完成，基础数据量' AS section;
SELECT
  (SELECT COUNT(*) FROM BookClass) AS class_count,
  (SELECT COUNT(*) FROM Publisher) AS publisher_count,
  (SELECT COUNT(*) FROM Book) AS book_count,
  (SELECT COUNT(*) FROM Reader) AS reader_count,
  (SELECT COUNT(*) FROM Borrow) AS borrow_count;
CALL run_integrity_test('插入BookClass重复主键: ("001","机械类")',
  'INSERT INTO BookClass VALUES ("001","机械类");');
```

运行输出: outputs/06_bookdb_integrity.txt

+-----+ section +-----+				
实验6：完整性约束定义完成，基础数据量 +-----+				
+-----+ class_count publisher_count book_count reader_count borrow_count +-----+				
5 5 20 10 50 +-----+				
+-----+				
section +-----+				
实验6：完整性检查结果 +-----+				
+-----+				
+-----+ id testItem result message +-----+				
+-----+				
1 插入BookClass重复主键: ('001','机械类') 被完整性约束拦截 该操作违反主键、外键或CHECK约束，已被MySQL完整性机制拦截				
2 插入Reader重复编号或身份证: R2020001 被完整性约束拦截 该操作违反主键、外键或CHECK约束，已被MySQL完整性机制拦截				
3 插入Borrow不存在的读者R2021003或日期检查 执行成功 执行成功				
4 删除已有借阅记录的读者R2020001 被完整性约束拦截 该操作违反主键、外键或CHECK约束，已被MySQL完整性机制拦截				
5 删除无借阅记录的读者R2022003 执行成功 执行成功				
6 将R2020002借阅B202001004日期改为2021-04-22 被完整性约束拦截 该操作违反主键、外键或CHECK约束，已被MySQL完整性机制拦截				
7 将R2020002最大借书数量改为9 被完整性约束拦截 该操作违反主键、外键或CHECK约束，已被MySQL完整性机制拦截				
8 数据库系统原理价格上调30元 被完整性约束拦截 该操作违反主键、外键或CHECK约束，已被MySQL完整性机制拦截				
+-----+				

五、实验心得

完整性约束能把业务规则固定在数据库层，减少应用程序遗漏校验造成的数据错误。

桂林理工大学

实验报告

班级 软件工程 3 班

学号 3242052051355

姓名 洪耀东

实验名称 实验七 执行计划

日期 2026 年 6 月 14 日

一、实验目的：

1. 掌握使用 EXPLAIN 查看 SQL 查询执行计划的方法。
2. 理解访问类型、候选索引、实际索引、扫描行数和 Extra 信息对 SQL 优化的意义。

二、实验环境：

1. Windows，本机 MySQL80 服务。
2. MySQL 8.0.41，OrderDB 数据库。
3. mysql 命令行客户端、PowerShell。

三、实验内容：

1.1 索引准备

为商品名称、订单明细商品号、订单号、订单主表员工号和客户号建立索引。

1.2 执行计划分析

查看“酷睿四核 i5-6500”销售情况查询和每个员工销售记录查询的执行计划。

四、实验练习

1.1 源码文件

D:\sandbox\MYSQL_homework\sql\07_execution_plan.sql

1.2 运行结果

EXPLAIN 结果显示查询使用了相关索引，并输出对应销售查询结果。

运行截图 7.1:

源码与真实运行输出截图

```

SET NAMES utf8mb4;
USE OrderDB;
CREATE INDEX idx_product_name ON Product(productName);
CREATE INDEX idx_orderdetail_product ON OrderDetail(productNo);
CREATE INDEX idx_orderdetail_order ON OrderDetail(orderNo);
CREATE INDEX idx_ordermaster_employee ON OrderMaster(employeeNo);
CREATE INDEX idx_ordermaster_customer ON OrderMaster(customerNo);
SELECT '实验7：查询1执行计划-酷睿四核i5-6500销售情况' AS section;
EXPLAIN FORMAT=TRADITIONAL
SELECT e.employeeName,
       CASE e.sex WHEN 'M' THEN '男' ELSE '女' END AS sex,
       om.orderDate,
       od.quantity,
       od.quantity * od.price AS amount
FROM OrderDetail od
JOIN OrderMaster om ON od.orderNo = om.orderNo
JOIN Employee e ON om.employeeNo = e.employeeNo
WHERE od.productNo = (
  SELECT productNo
  FROM Product
  WHERE productName = '酷睿四核i5-6500'
);
SELECT e.employeeName,
       CASE e.sex WHEN 'M' THEN '男' ELSE '女' END AS sex,
       om.orderDate,
       od.quantity,
       od.quantity * od.price AS amount
FROM OrderDetail od
JOIN OrderMaster om ON od.orderNo = om.orderNo
JOIN Employee e ON om.employeeNo = e.employeeNo
WHERE od.productNo = (
  SELECT productNo
  FROM Product
  WHERE productName = '酷睿四核i5-6500'
);
SELECT '实验7：查询2执行计划-每个员工销售记录' AS section;
EXPLAIN FORMAT=TRADITIONAL
SELECT e.employeeNo,
       e.employeeName,
       CASE e.sex WHEN 'M' THEN '男' ELSE '女' END AS sex,
       p.productName,
       od.quantity,
       od.price,
       od.quantity * od.price AS amount,
       DATE_FORMAT(om.orderDate, '%Y-%m-%d') AS orderDate
FROM Employee e
JOIN OrderMaster om ON e.employeeNo = om.employeeNo
JOIN OrderDetail od ON om.orderNo = od.orderNo
JOIN Product p ON od.productNo = p.productNo
ORDER BY e.employeeNo;
SELECT e.employeeNo,
       e.employeeName,
       CASE e.sex WHEN 'M' THEN '男' ELSE '女' END AS sex,
       p.productName,
       od.quantity,
       od.price,
       od.quantity * od.price AS amount,
       DATE_FORMAT(om.orderDate, '%Y-%m-%d') AS orderDate
FROM Employee e
JOIN OrderMaster om ON e.employeeNo = om.employeeNo
JOIN OrderDetail od ON om.orderNo = od.orderNo
JOIN Product p ON od.productNo = p.productNo
ORDER BY e.employeeNo;
SELECT '实验7问题解答' AS section;
SELECT '优化原则：只取必要列；优先使用等值连接与可索引谓词；为连接列、筛选列和排序列建立合适索引；避免在索引列上包裹函数；用EXPLAIN观察访问类型。快选索引、扫描行数和Extra信息，再根据瓶颈调整SQL或索引。' AS answer;

```

```
+-----+
| section |
+-----+
|实验7：查询1执行计划-酷睿四核5-6500销售情况 |
+-----+
+-----+
+-----+
+-----+
+-----+
+-----+
| id | select_type | table | partitions | type | possible keys | key | |
|---|---|---|---|---|---|---|---|
|    |      |key_len| ref       |     | rows | filtered | Extra   |
+-----+
+-----+
| 1 | PRIMARY | od | NULL | ref | PRIMARY,idx_orderdetail_product,idx_orderdetail_order | idx_orderdetail_product | 36 | const | 3 | 100.00 | Using where |
+-----+
| 1 | PRIMARY | om | NULL | eq_ref | PRIMARY,idx_ordermaster_employee | PRIMARY | 36 | orderdb.od.orderNo | 1 | 100.00 | Using where |
+-----+
| 1 | PRIMARY | e | NULL | eq_ref | PRIMARY | PRIMARY | 32 | orderdb.om.employeeNo | 1 | 100.00 | NULL |
+-----+
| 2 | SUBQUERY | Product | NULL | ref | idx_product_name | idx_product_name | 322 | const | 1 | 100.00 | Using index |
+-----+
+-----+
+-----+
| employeeName | sex | orderDate | quantity | amount |
+-----+
|李强 |男 |2020-03-05 |2 |2900.00 |
|李强 |男 |2020-04-02 |1 |1450.00 |
|张梅 |女 |2021-01-22 |2 |2900.00 |
+-----+
+-----+
| section |
+-----+
|实验7：查询2执行计划-每个员工销售记录 |
+-----+
+-----+
+-----+
+-----+
+-----+
+-----+
| id | select_type | table | partitions | type | possible keys | key | |
|---|---|---|---|---|---|---|---|
|    |      |key_len| ref       |     | rows | filtered | Extra   |
+-----+
+-----+
+-----+
| 1 | SIMPLE | om | NULL | ALL | PRIMARY,idx_ordermaster_employee | PRIMARY | 36 | NULL | NULL | 16 | 100.00 | Using where; Using temporary; Using filesort |
+-----+
| 1 | SIMPLE | e | NULL | eq_ref | PRIMARY | PRIMARY | 32 | orderdb.om.employeeNo | 1 | 100.00 | NULL |
+-----+
| 1 | SIMPLE | od | NULL | ref | PRIMARY,idx_orderdetail_product,idx_orderdetail_order | idx_orderdetail_order | 36 | orderdb.od.orderNo | 2 | 100.00 | NULL |
+-----+
| 1 | SIMPLE | p | NULL | eq_ref | PRIMARY | PRIMARY | 36 | orderdb.od.productNo | 1 | 100.00 | NULL |
+-----+
+-----+
+-----+
+-----+
+-----+
+-----+
| employeeNo | employeeName | sex | productName | quantity | price | amount |
|orderDate |
+-----+
+-----+
+E2019001 |李强 |男 |酷睿四核5-6500 |2 |1450.00 |2900.00 |2020-03-05 |
+E2019001 |李强 |男 |金士顿16G内存 |5 |329.00 |1645.00 |2020-03-05 |
+E2019001 |李强 |男 |TP-LINK千兆路由器 |4 |199.00 |796.00 |2020-03-05 |
+E2019001 |李强 |男 |酷睿四核5-6500 |1 |1450.00 |1450.00 |2020-04-02 |
+E2019001 |李强 |男 |HP激光打印机 |2 |1299.00 |2598.00 |2020-04-02 |
+E2019001 |李强 |男 |罗技无线鼠标 |1 |129.00 |129.00 |2026-06-14 |
+E2019001 |李强 |男 |酷睿四核I7-7700k |1 |2499.00 |2499.00 |2020-07-20 |
```

SQL 优化应先保证结果正确，再利用 EXPLAIN 观察是否走索引、是否产生临时表或文件排序，并据此调整 SQL 或索引。

桂林理工大学

实验报告

班级 软件工程 3 班

学号 3242052051355

姓名 洪耀东

实验名称 实验八 事务处理

日期 2026 年 6 月 14 日

一、实验目的：

1. 理解事务提交、回滚和隔离级别的作用。
2. 掌握 START TRANSACTION、COMMIT 以及多连接并发验证方法。

二、实验环境：

1. Windows，本机 MySQL80 服务。
2. MySQL 8.0.41，OrderDB 数据库。
3. mysql 命令行客户端、PowerShell、Python、PyMySQL。

三、实验内容：

1.1 事务提交

定义事务注册新客户，并定义事务登记 2022 年 8 月 16 日的新订单，新订单享受 2%优惠。

1.2 事务更新

定义事务处理业务员 E2020003 离职，解除订单引用后删除员工信息。

1.3 隔离级别

通过两个数据库连接模拟 T1 查询平均薪水、T2 插入新员工并提交，比较 READ COMMITTED 和 REPEATABLE READ 效果。

四、实验练习

1.1 源码文件

D:\sandbox\MYSQL_homework\sql\08_transactions.sql;

D:\sandbox\MYSQL_homework\scripts\run_transaction_isolation_demo.py

1.2 运行结果

新客户、新订单和离职处理事务执行成功；并发演示显示不同隔离级别下两次读取结果不同。

运行截图 8.1：

实验8 事务处理

源码与真实运行输出截图

代码: sql/08_transactions.sql SET NAMES utf8mb4; USE OrderDB; DELETE FROM OrderDetail WHERE orderNo = 'O20220816'; DELETE FROM OrderMaster WHERE orderNo = 'O20220816'; DELETE FROM Customer WHERE customerNo = 'C20220816'; SELECT '实验8: 事务1-新客户注册' AS section; START TRANSACTION; INSERT INTO Customer(customerNo, customerName, address, zip, phone) VALUES ('C20220816', '柳州新客户有限公司', '柳州市城中区', '545001', '0772-2160816'); COMMIT; SELECT customerNo, customerName, address FROM Customer WHERE customerNo = 'C20220816'; SELECT '实验8: 事务2-新订单登记并享受2%优惠' AS section; START TRANSACTION; INSERT INTO OrderMaster(orderNo, customerNo, employeeNo, orderDate) VALUES ('O20220816', 'C20220816', 'E2020004', '2022-08-16'); SELECT ROUND(productPrice * 0.98, 2) INTO @price_p20210003 FROM Product WHERE productNo = 'P20210003'; SELECT ROUND(productPrice * 0.98, 2) INTO @price_p20210002 FROM Product WHERE productNo = 'P20210002'; INSERT INTO OrderDetail(orderNo, productNo, quantity, price) VALUES ('O20220816', 'P20210003', 3, @price_p20210003); INSERT INTO OrderDetail(orderNo, productNo, quantity, price) VALUES ('O20220816', 'P20210002', 2, @price_p20210002); COMMIT; SELECT om.orderNo, om.customerNo, om.orderDate, om.orderSum FROM OrderMaster om WHERE om.orderNo = 'O20220816'; SELECT od.orderNo, od.productNo, p.productName, od.quantity, od.price, od.quantity * od.price AS amount FROM OrderDetail od JOIN Product p ON od.productNo = p.productNo WHERE od.orderNo = 'O20220816'; SELECT '实验8: 事务3-业务员E2020003离职, 删除相关信息' AS section; START TRANSACTION; UPDATE OrderMaster SET employeeNo = NULL WHERE employeeNo = 'E2020003'; DELETE FROM Employee WHERE employeeNo = 'E2020003'; COMMIT; SELECT COUNT(*) AS employee_E2020003_count FROM Employee WHERE employeeNo = 'E2020003'; SELECT orderNo, employeeNo FROM OrderMaster WHERE employeeNo IS NULL ORDER BY orderNo; SELECT '实验8: 隔离级别并发演示请运行 scripts/run_transaction_isolation_demo.py' AS section;	运行输出: outputs/08_transactions.txt +-----+ section +-----+ 实验8: 事务1-新客户注册 +-----+ +-----+-----+-----+ customerNo customerName address +-----+-----+-----+ C20220816 柳州新客户有限公司 柳州市城中区 +-----+-----+-----+ +-----+ section +-----+ 实验8: 事务2-新订单登记并享受2%优惠 +-----+ +-----+-----+-----+ orderNo customerNo orderDate orderSum +-----+-----+-----+ O20220816 C20220816 2022-08-16 847.70 +-----+-----+-----+ +-----+-----+-----+-----+ orderNo productNo productName quantity price amount +-----+-----+-----+-----+ O20220816 P20210002 小米手环7 2 234.22 468.44 O20220816 P20210003 罗技无线鼠标 3 126.42 379.26 +-----+-----+-----+-----+ +-----+ section +-----+ 实验8: 事务3-业务员E2020003离职, 删除相关信息 +-----+ +-----+ employee_E2020003_count +-----+ 0 +-----+ +-----+ orderNo employeeNo +-----+ O20200003 NULL O20200008 NULL O20210003 NULL +-----+ +-----+ section +-----+ 实验8: 隔离级别并发演示请运行 scripts/run_transaction_isolation_demo.py +-----+
---	--

运行截图 8.2:

源码与真实运行输出截图

读

实验8：事务隔离级别并发演示

T1负责两次查询Employee平均薪水，T2在两次查询之间插入两名高薪员工并提交。

READ COMMITTED	6885.71	8077.78	第二次读到T2提交的新数据
REPEATABLE READ	6885.71	6885.71	同一事务内保持一致读

结论: READ COMMITTED会在第二次查询时看到其他事务已提交的数据;
REPEATABLE READ在同一事务内保持第一次读到的快照, 因此两次结果一致。

事务隔离级别会影响同一事务内多次查询能否看到其他事务提交的数据，实际应用中需要根据一致性要求选择合适级别。